

◆ DEVELOPER BUILD · SHIP

CI/CD Build-Failure Triage Agent

what we're going to build

A LangGraph agent that reads your **real** failed GitHub Actions runs, diagnoses the root cause from the real logs with gpt-4o, and posts a triage on the commit behind a **human-approval gate**. This deck is the engineering plan — architecture, the graph, the files, and how to run it.

LangGraph

LangChain tools

gpt-4o

GitHub REST API

Streamlit

GitHub Actions

What **we**, the developers, are building

READ

Live GitHub Actions access

Read-only over the GitHub REST API — list failed runs, find the failing job/step, pull the **real logs**. No mock data.

REASON

A LangGraph agent

A state machine that investigates in a ReAct loop, then emits a **structured** triage: category · root cause · evidence · fix · confidence.

GATE

Human-in-the-loop

The graph **pauses** at `interrupt()` for approval before anything is posted. Dry-run by default.

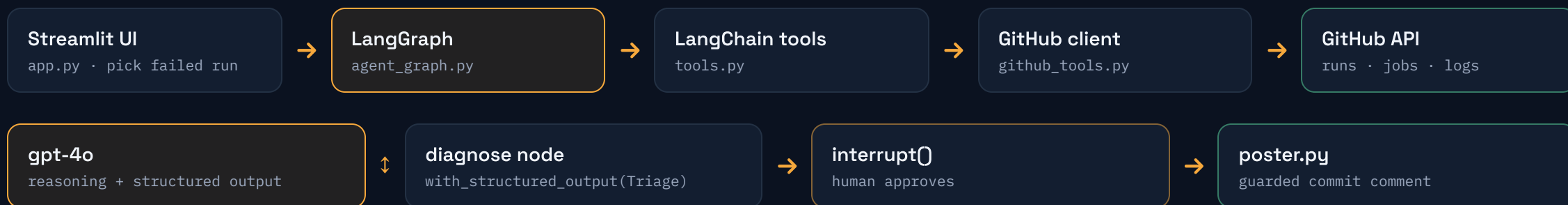
ACT (guarded)

One safe write

Post the triage as a **commit comment**. That's the only write — no code changes, no arbitrary API calls.

Design rule: the LLM only reads and reasons. Exactly one module can write to GitHub, and only after a human clicks approve. Everything else is read-only.

How the pieces fit together



Data flow: UI → graph → tools read the real runs + logs → gpt-4o diagnoses → graph pauses for approval → the guarded module posts one comment. State lives in a MemorySaver checkpoint.

The agent is a **state machine**

```
# agent_graph.py – nodes & edges
g = StateGraph(State)
g.add_node("investigate", investigate) # LLM+tools
g.add_node("tools", ToolNode(TOOLS))
g.add_node("diagnose", diagnose) # structured
g.add_node("approval_gate", approval_gate)
g.add_node("post", post)

g.add_conditional_edges("investigate",
    tools_condition, {"tools": "tools", END: "diagnose"})
g.add_edge("tools", "investigate") # ReAct loop
g.add_edge("diagnose", "approval_gate")
g.compile(checkpointer=MemorySaver())
```

- ▶ **investigate** $\rightleftharpoons$ **tools** — the agent reads jobs + real logs, looping until it has evidence.
- ▶ **diagnose** — one structured LLM call $\rightarrow$ a typed **Triage** (Pydantic).
- ▶ **approval_gate** — **interrupt()** pauses the graph; the UI resumes with **Command(resume=...)**.
- ▶ **post** — posts the triage comment, dry-run unless **execute** is on.

The files we write — and **real** failures to triage

FILE	RESPONSIBILITY
<code>agent_graph.py</code>	the LangGraph orchestration
<code>tools.py</code>	read-only LangChain tools
<code>github_tools.py</code>	live GitHub REST access + logs
<code>poster.py</code>	guarded commit-comment write
<code>app.py</code>	Streamlit UI drives the graph
<code>live-ci/*.yaml</code>	real failing workflows

Real failures we deploy to CI on purpose

Real GitHub Actions runs in `live-ci/` — the agent reads their real logs, nothing is canned.

- ▶ **test_failure** — `add()` subtracts → `assert -1 == 5`
- ▶ **dependency** — `requests==99.99.99` doesn't exist
- ▶ extend: compile error · lint · flaky · OOM · timeout

Verified: both real runs triaged at **High** confidence with the exact evidence line — and a real triage comment posted on the commit.

Run the **live demo**

- 1 **Install** deps into a venv.
- 2 **Add your OpenAI key** to `.env`.
- 3 **Authenticate GitHub** — `gh auth login` (or a token).
- 4 **Make real failures** — push the `live-ci/` workflows.
- 5 **Launch** — Streamlit auto-lists the failed runs.

Needs: Python 3.10+ · OpenAI key · a GitHub repo with Actions. Token scopes: `Actions:Read` · `Contents:Read` · `Commit statuses:Write`.

```
# 1-2 · install + key
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp env.sample .env    # paste OPENAI_API_KEY

# 3 · GitHub auth (either one)
gh auth login          # agent uses this token
# or: GITHUB_TOKEN=... in .env

# 4 · create real failed runs
cp live-ci/*.yaml .github/workflows/
git add -A && git commit -m "ci" && git push

# 5 · run the agent
streamlit run app.py    # → pick a run → triage → approve → post
```